

19. Webové aplikace - výběr technologií

Kritéria

- Co to bude za projekt – velikostně, hardwarové požadavky, využití, zabezpečení
- Time-to-market – za jak krátkou dobu musí být aplikace hotová
- Tým – jaké technologie ovládá, jaké naopak neovládá
- Rozpočet – co vše si můžeme dovolit (např. AWS cloud nebo sdílený hosting)

Frontend

- Jak má aplikace vypadat, reagovat a aby byla lehce přístupná těm kdo jí bude používat
- Základní kombinace:
 - HTML/CSS/JS – jednoduché weby, rychlé načítání, žádné závislosti, nejlepší pro SEO
 - Frameworky – složitější weby, single page apps, interaktivní prvky, často měnící se stavy, nutnost ovládání daného frameworku (např. React, Vue.js, Angular)
 - Meta-frameworky – výhody SPA a klasického webu díky server side rendering, brutální zlepšení výkonu, načítání, SEO (optimalizace)

Backend

- Mozek aplikace, co jak bude fungovat, logika aplikace, všechny funkční prvky, komunikace s DB, bezpečnostní logika
- Výběr:
 - Node.js (Express) – asynchronní, tisíce spojení najednou, psaní frontedu i backendu v JS, dobré např. pro API a chaty, rychlá křivka učení, JS stejně většina ovládá
 - PHP (Laravel) / Python (Django) – tradiční, robustní, PHP stále na většině webů, python pro AI a práci s velkými daty
 - Java / C# (.NET) - bezpečné, striktní, bankovní systémy, korporátní portály, vysoká křivka učení

Databáze

- Jaká data bude aplikace ukládat, v jakém formátu, jak často k nim bude přistupovat, kolik těch dat bude, jaké operace bude muset DB vykonávat
- Výběr:
 - Relační DB (PostgreSQL, MySQL, MariaDB, ...) – tabulky, pevné vazby, zaručení ACID pravidel (spolehlivost provedení transakcí), využití: eshopy, finance, účetnictví (vhodné pro ně úplně základní projekty)
 - Nerelační DB (MongoDB, Redis) – flexibilní dokumenty např v JSONu, žádné pevné schéma, extrémně rychlé a škálovatelné, využití: logy, analytika, real-time data, cache (data, která nemají moc vaby na jiné soubory)

Architektura a deployment

- Jak se aplikace udělá veřejnou, kde bude nahraná, jak bude strukturována
- Monolith vs Mikroslužby:
 - Monolith je aplikace v jednom souboru, snadný začátek, rychlé a jednoduché, těžké na udržení při škálování a rozšiřování, časem hodně těžko udržitelné pro velké weby
 - Mikroslužby jsou rozdělení webu na více menších nezávislých částí, složitější ze začátku ale do budoucna obrovská výhoda pro škálování, must have pro velké aplikace
- Docker – standard, zabalení app i s jejím prostředím do virtuálního kontejneru, řešení problému “Ale mě to na pc fungovalo!”
- Hosting:
 - Fyzický server – drahé, nutná znalost konfigurace a údržby, vysoká flexibilita, ideální pro velké a náročné aplikace
 - VPS – jednoduché, relativně levné, stále nutnost umět konfigurovat, nemusí být tak výkonné, mnoho druhů a stále velice flexibilní
 - Sdílený hosting – levné, jednoduché, nahrání souborů a db, né příliš výkonné, ideální pro statické stránky
 - CI/CD – automatizovaný deployment, po nahrání kódu na github se spustí testy, zabalí se do docker obrazu a nasadí se na server (např. GitHub actions)
 - Cloud – může být hodně drahé (metoda pay-as-you-go – platí se jen za to co se reálně spotřebuje), extrémní škálovatelnost, ideální pro velké výkyvy využití (např. Sezoní)

Webové servery

- Co reálně bude zpřístupňovat ten web softwarově na serveru
- Výběr:
 - Apache HTTP Server – starší klasika, rozumí si s PHP, soubor .htaccess pro správu přístupů pro každou složku zvlášť, dnes již zastaralé
 - Nginx – moderní, extrémně populární, asynchronní architektura, zvládá obrovské množství požadavků a uživatelů najednou, malá spotřeba paměti, vyniká v servírování statických souborů, funguje jako reverzní proxy (postaví se před app a odfiltruje škodlivý provoz), může fungovat jako load balancer